Tree Methods I

Math 243

Michael Pearce

Outline

In today's class, we will...

- Start a 3-week unit on "Trees"
- Discuss regression trees as a non-parametric model

Section 1

Movies!

Activity: Guessing movies, TV shows, or books

1) Think of one of your favorite movies, TV shows, or books (try to pick something that is decently well-known).

2) With a partner, ask each other Yes-or-No questions to deduce your partner's favorite movie/show/book.

3) Write down the questions you asked each other

Reflection

With your partner, reflect on the following questions:

- Q: What was your strategy in asking questions?
- Q: What makes an effective question?

Reflection

With your partner, reflect on the following questions:

- Q: What was your strategy in asking questions?
- Q: What makes an effective question?

Some good guidelines:

- Later questions should be based on answers to earlier questions
- Early questions are general, later questions are more specific
- Good questions might separate data into roughly equal sizes, and observations within groups should be relatively similar

We just did a math

Each of your guessing algorithms forms a (partial) decision tree

We just did a math

Each of your guessing algorithms forms a (partial) decision tree

- Yes/No questions represent a branching point or node, and partition the movies/shows/books
- After identifying a small enough set of potential movies/shows/books, you make a guess (or a "prediction")

We just did a math

Each of your guessing algorithms forms a (partial) decision tree

- Yes/No questions represent a branching point or node, and partition the movies/shows/books
- After identifying a small enough set of potential movies/shows/books, you make a guess (or a "prediction")

Classification and Regression Trees (CART) do the same thing:

- Partition the predictor space into subsections
- Given a test point x_0, find which subsection it falls in
- predict the "average" training y_i in corresponding subsection

We'll start with Regression Trees (quantitative Y), and then move Classification trees

Section 2

Regression Trees

Regression Trees

Regression Trees: Partition data into subsections/groups that are similar on predictors. Then, make predictions based on average response value in each group.

Regression Trees

Regression Trees: Partition data into subsections/groups that are similar on predictors. Then, make predictions based on average response value in each group.

Most common technique is the Classification and Regression Tree (CART) method.

1. Starting with the entire data set S, consider every potential "cut-off" point c for every predictor X_k that splits S into two groups:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

Regression Trees

Regression Trees: Partition data into subsections/groups that are similar on predictors. Then, make predictions based on average response value in each group.

Most common technique is the Classification and Regression Tree (CART) method.

1. Starting with the entire data set S, consider every potential "cut-off" point c for every predictor X_k that splits S into two groups:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

and choose the optimal S_1 and S_2 (corresponding to the optimal X_k and c) which minimizes the sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

Regression Trees

Regression Trees: Partition data into subsections/groups that are similar on predictors. Then, make predictions based on average response value in each group.

Most common technique is the Classification and Regression Tree (CART) method.

1. Starting with the entire data set S, consider every potential "cut-off" point c for every predictor X_k that splits S into two groups:

$$S_1 = \text{observations with } X_k \leq c$$
$$S_2 = \text{observations with } X_k > c$$

and choose the optimal S_1 and S_2 (corresponding to the optimal X_k and c) which minimizes the sum of squared error:

$$\text{SSE} = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

where $\bar{y}_1$ and $\bar{y}_2$ are the average response value in groups S_1 and S_2

2. Repeat Step 1 for each of the two groups S_1 and S_2.
3. The method continues splitting groups until each subdivision has few observation (or another predetermined stopping condition is met)

Trees on Trees

We'll use a subset of the `pdxTrees` dataset from the `pdxTrees` package (McConville, Caldwell, and Horton)



- The data was collected by the Portland Parks and Rec's *Urban Forestry Tree Inventory Project*.
- The Tree Inventory Project has gathered data on Portland trees since 2010, collecting this data in the summer months with a team of over 1,300 volunteers and city employees.

pdxTrees Data

The pdxTrees dataset is very large, so the package provides "helper" loading functions:

- `get_pdxTrees_parks()` pulls data on 25,534 trees from 174 Portland parks
- `get_pdxTrees_streets()` pulls data on 218,602 trees along Portland streets

pdxTrees Data

The pdxTrees dataset is very large, so the package provides "helper" loading functions:

- `get_pdxTrees_parks()` pulls data on 25,534 trees from 174 Portland parks
- `get_pdxTrees_streets()` pulls data on 218,602 trees along Portland streets

We'll just focus on trees in parks near Reed:

```
library(pdxTrees)
my_pdxTrees <- get_pdxTrees_parks(park = c("Kenilworth Park", "Westmoreland Park",
                                           "Woodstock Park","Berkeley Park", "Powell Park"))
```

pdxTrees Data

The pdxTrees dataset is very large, so the package provides "helper" loading functions:

- `get_pdxTrees_parks()` pulls data on 25,534 trees from 174 Portland parks
- `get_pdxTrees_streets()` pulls data on 218,602 trees along Portland streets

We'll just focus on trees in parks near Reed:

```
library(pdxTrees)
my_pdxTrees <- get_pdxTrees_parks(park = c("Kenilworth Park", "Westmoreland Park",
                                           "Woodstock Park","Berkeley Park", "Powell Park"))
```

How many observations?

```
dim(my_pdxTrees)
```

```
## [1] 1039   34
```

pdxTrees Data

The pdxTrees dataset is very large, so the package provides "helper" loading functions:

- `get_pdxTrees_parks()` pulls data on 25,534 trees from 174 Portland parks
- `get_pdxTrees_streets()` pulls data on 218,602 trees along Portland streets

We'll just focus on trees in parks near Reed:

```
library(pdxTrees)
my_pdxTrees <- get_pdxTrees_parks(park = c("Kenilworth Park", "Westmoreland Park",
                                           "Woodstock Park","Berkeley Park", "Powell Park"))
```

How many observations?

```
dim(my_pdxTrees)
```

```
## [1] 1039   34
```

What variables are present?

```
names(my_pdxTrees)
```

```
##  [1] "Longitude"                "Latitude"
##  [3] "UserID"                   "Genus"
##  [5] "Family"                   "DBH"
##  [7] "Inventory_Date"           "Species"
##  [9] "Common_Name"              "Condition"
## [11] "Tree_Height"              "Crown_Width_NS"
## [13] "Crown_Width_EW"           "Crown_Base_Height"
## [15] "Collected_By"             "Park"
## [17] "Scientific_Name"          "Functional_Type"
## [19] "Mature_Size"              "Native"
## [21] "Edible"                   "Nuisance"
## [23] "Structural_Value"         "Carbon_Storage_lb"
## [25] "Carbon_Storage_value"     "Carbon_Sequestration_lb"
## [27] "Carbon_Sequestration_value" "Stormwater_ft"
## [29] "Stormwater_value"         "Pollution_Removal_value"
## [31] "Pollution_Removal_oz"     "Total_Annual_Services"
## [33] "Origin"                   "Species_Factoid"
```

Carbon Sequestration

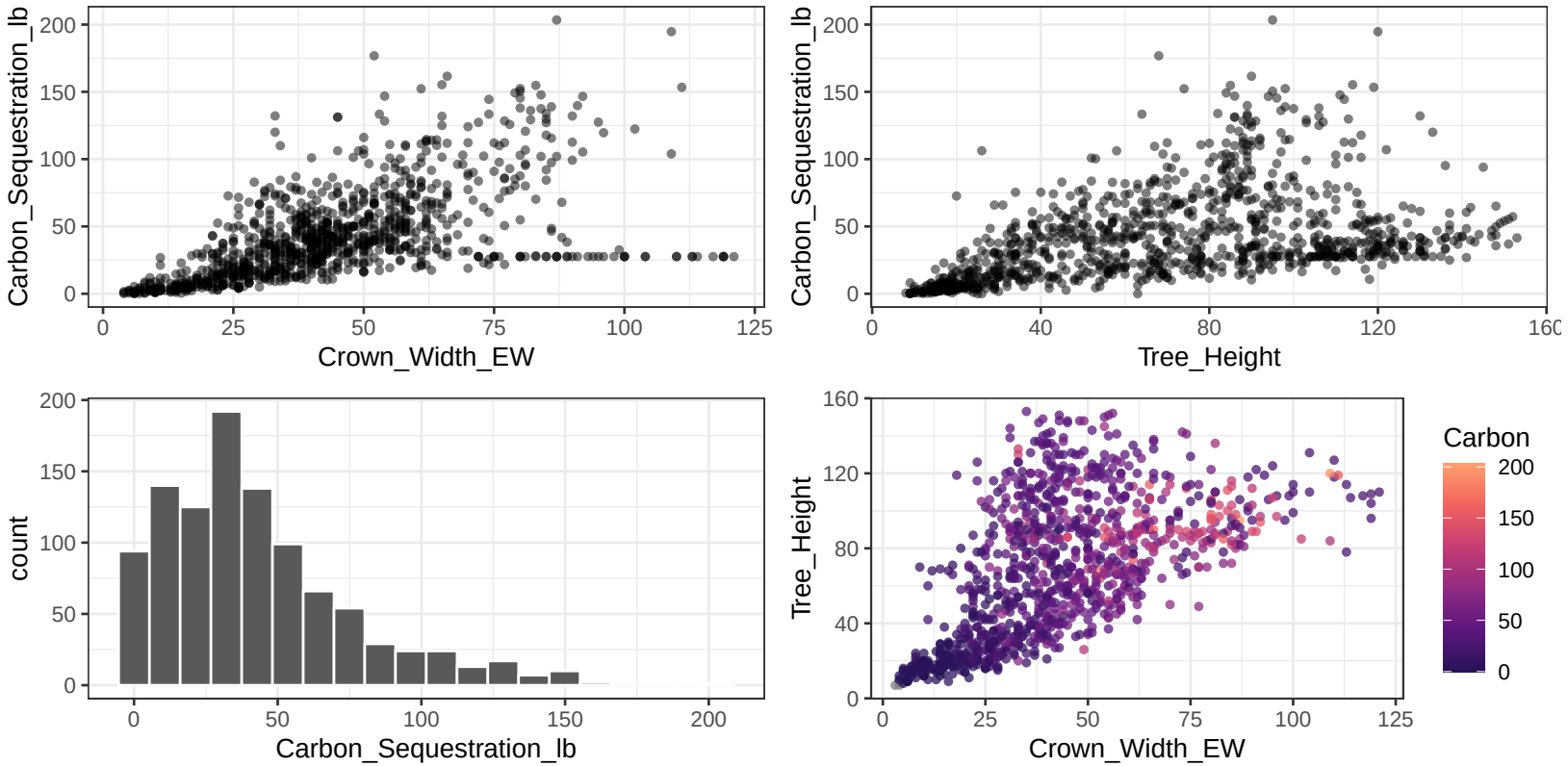
Goal: Let's predict Carbon Sequestration based on other tree features

- `Carbon_Sequestration_lb`: Amount of carbon (in lbs.) removed from the atmosphere by the tree, annually

Carbon Sequestration

Goal: Let's predict Carbon Sequestration based on other tree features

- `Carbon_Sequestration_lb`: Amount of carbon (in lbs.) removed from the atmosphere by the tree, annually
- Predictors: `Tree_Height` and `Crown_Width_EW`



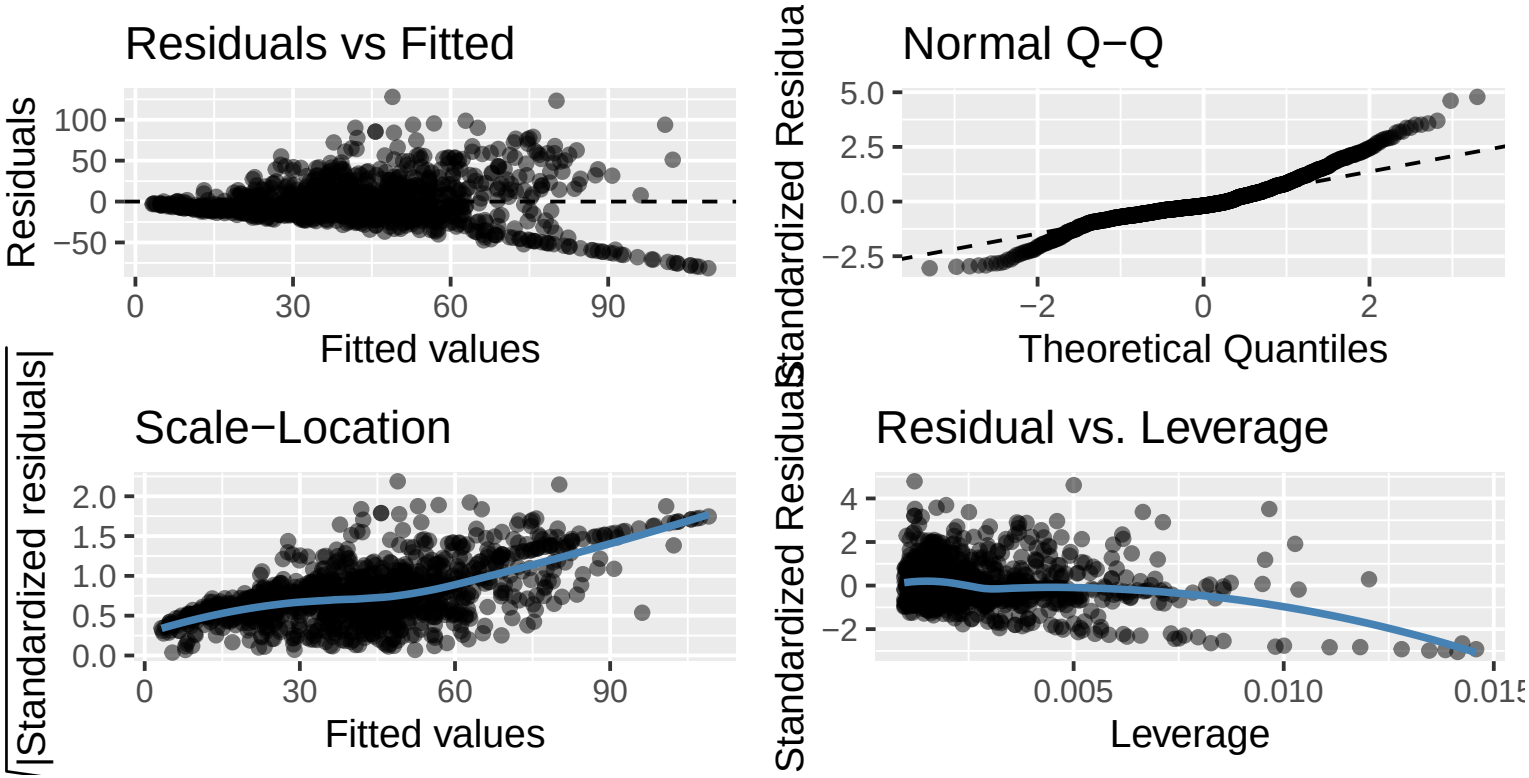
An Old Friend

This seems like a good time to implement linear regression:

```
tree_lm<-lm(Carbon_Sequestration_lb ~Crown_Width_EW + Tree_Height, data=my_pdxTrees)
summary(tree_lm)
```

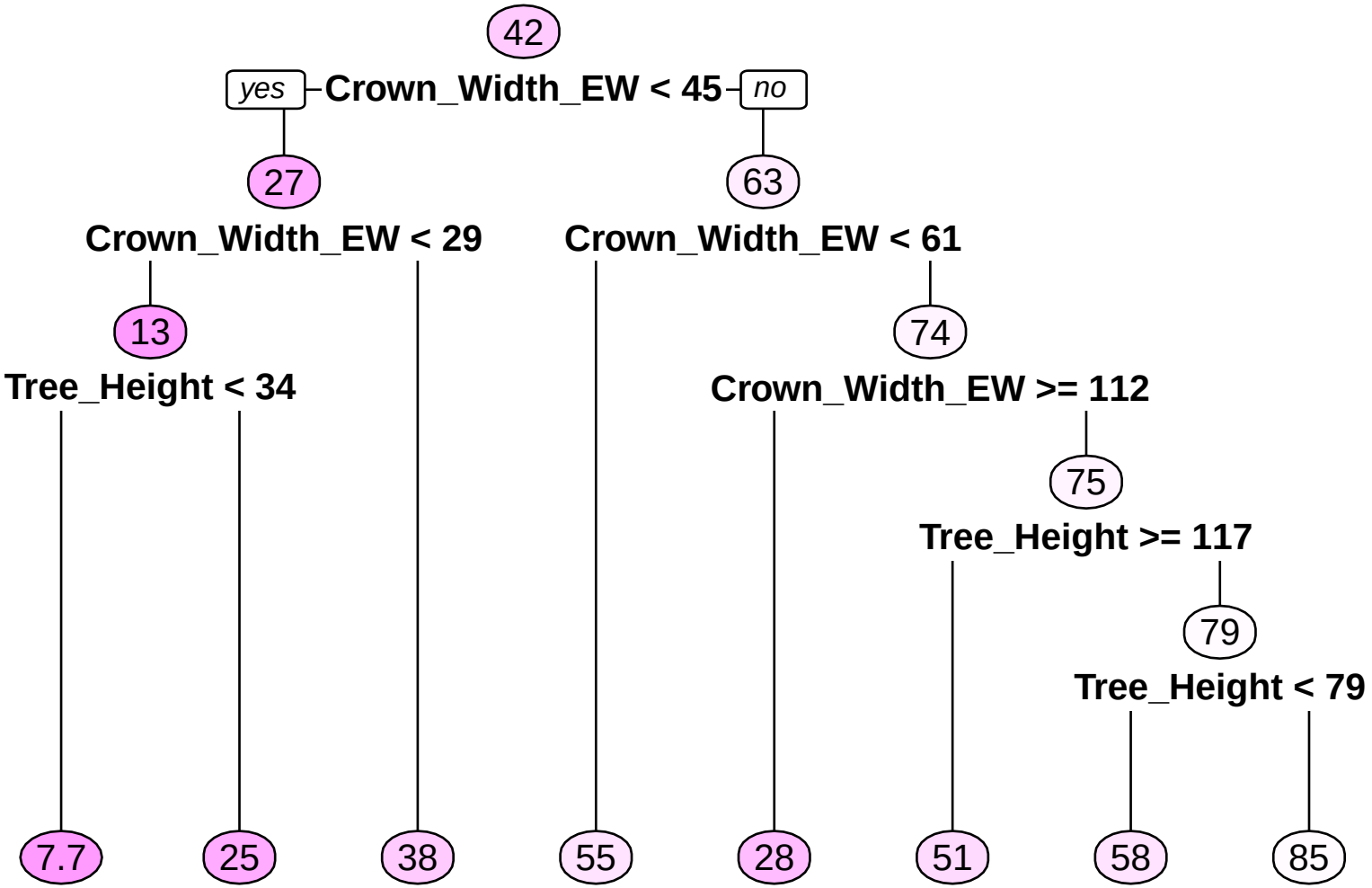
```
##
## Call:
## lm(formula = Carbon_Sequestration_lb ~ Crown_Width_EW + Tree_Height,
##     data = my_pdxTrees)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -81.46 -14.03  -4.92  11.51 127.87
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.06948    2.02087  -0.529    0.597
## Crown_Width_EW  0.79289    0.04624  17.146  < 2e-16 ***
## Tree_Height     0.12897    0.02820   4.573 5.38e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 26.74 on 1031 degrees of freedom
##   (5 observations deleted due to missingness)
## Multiple R-squared:  0.3699, Adjusted R-squared:  0.3687
## F-statistic: 302.7 on 2 and 1031 DF,  p-value: < 2.2e-16
```

Diagnostic Plots



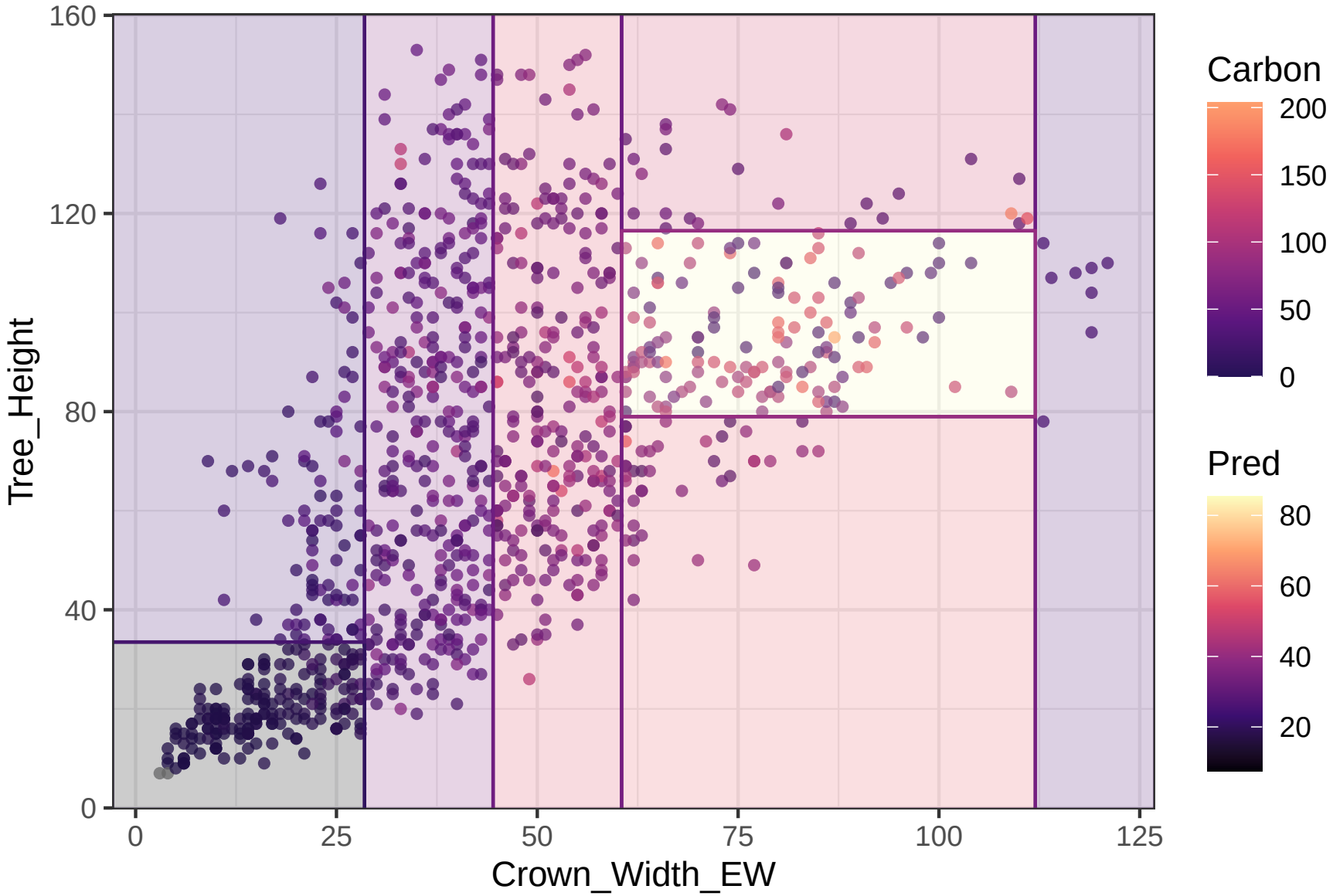
Some concerns here...

Regression Tree

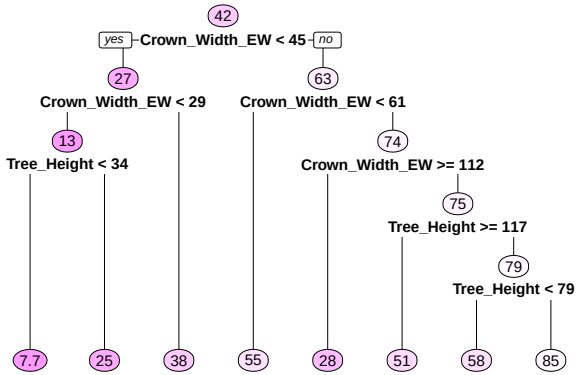


"Leaves" at the bottom of the tree are the predictions

Another Visualization



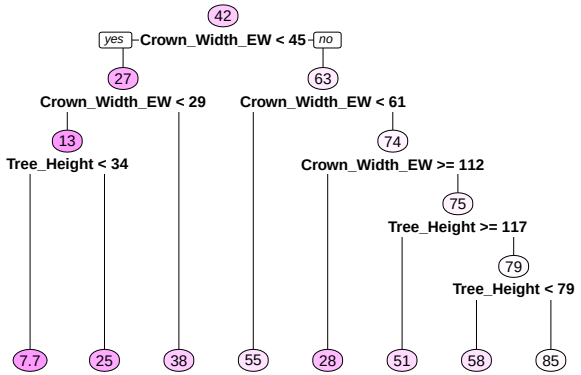
Interpretation



Tree splits on `Crown_Width_EW` first, then on `Tree_Height`

- `Crown_Width_EW` is the most important factor (first few splits!)
- `Tree_Height` is still important to account for (later splits!)

Interpretation



Tree splits on `Crown_Width_EW` first, then on `Tree_Height`

- `Crown_Width_EW` is the most important factor (first few splits!)
- `Tree_Height` is still important to account for (later splits!)

What kinds of trees have the highest/lowest predictions?

- Lowest: Crown_Width_EW < 29 and Tree_Height < 34
- Highest: Crown_Width_EW $\in [61, 112)$ and Tree_Height $\in [79, 117)$

Tree Accuracy

Let's create a test set consisting of parks further from Reed:

```
my_pdxTrees_test <- get_pdxTrees_parks(park = c("Mt Scott Park", "Glenwood Park"))
```

Tree Accuracy

Let's create a test set consisting of parks further from Reed:

```
my_pdxTrees_test <- get_pdxTrees_parks(park = c("Mt Scott Park", "Glenwood Park"))
```

- let's compare the test root MSE (rMSE) for the regression tree and linear regression:

$$\text{rMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Tree Accuracy

Let's create a test set consisting of parks further from Reed:

```
my_pdxTrees_test <- get_pdxTrees_parks(park = c("Mt Scott Park", "Glenwood Park"))
```

- let's compare the test root MSE (rMSE) for the regression tree and linear regression:

$$\text{rMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

```
##   Tree_rMSE
## 1  16.35376
```

```
##    lm_rMSE
## 1 30.45332
```

- The Regression Tree does better!

Tree Accuracy

Let's create a test set consisting of parks further from Reed:

```
my_pdxTrees_test <- get_pdxTrees_parks(park = c("Mt Scott Park", "Glenwood Park"))
```

- let's compare the test root MSE (rMSE) for the regression tree and linear regression:

$$\text{rMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

```
##   Tree_rMSE
## 1  16.35376
```

```
##    lm_rMSE
## 1 30.45332
```

- The Regression Tree does better!

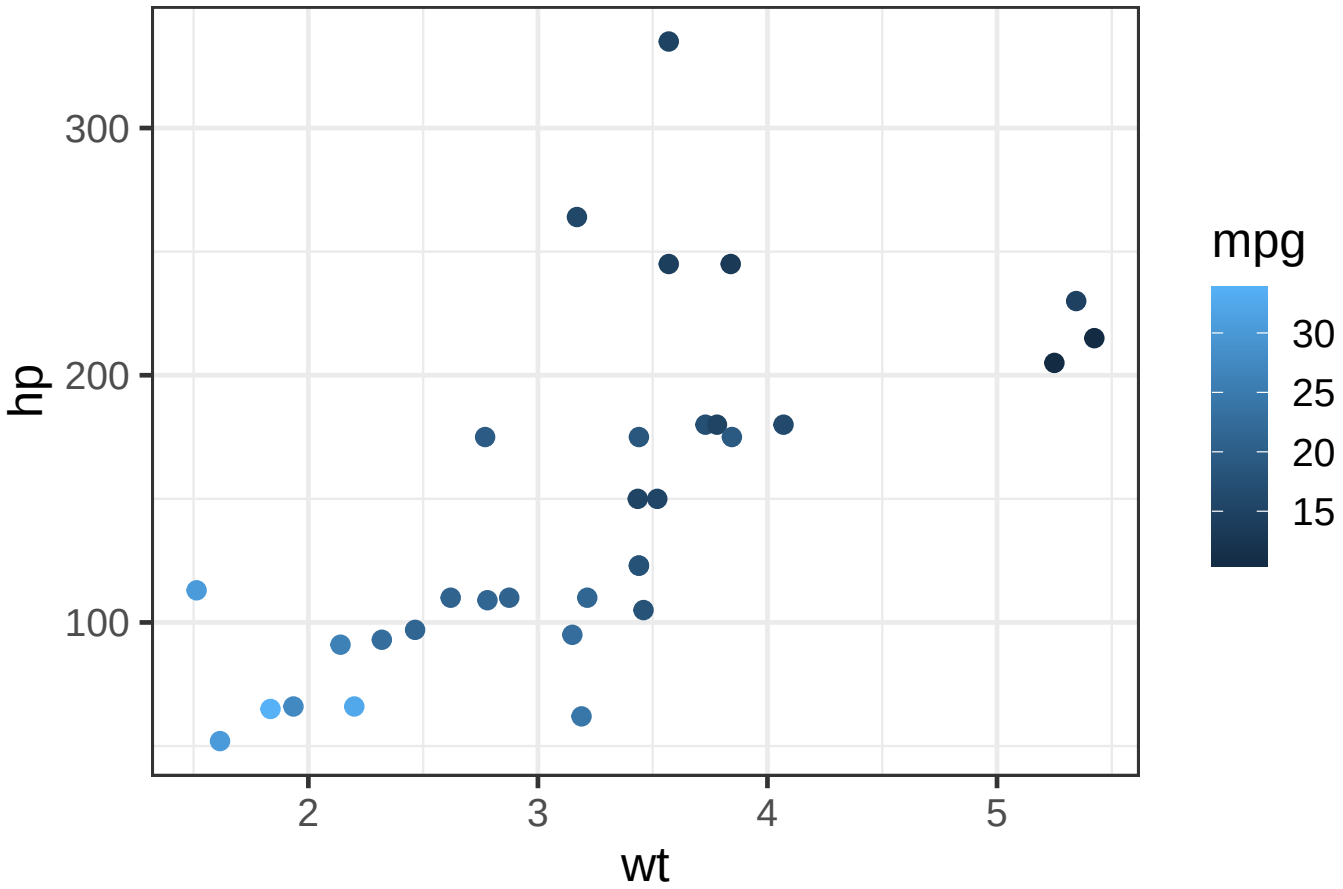
Moving forward, we'll often find it to be the case that trees will perform better than a linear model

- Q: Nevertheless, what are some downsides to the tree model?

Extra Practice

The `mtcars` dataset gives the MPG (`mpg`), horse-power (`hp`), and weight (`wt`) for 32 car models.

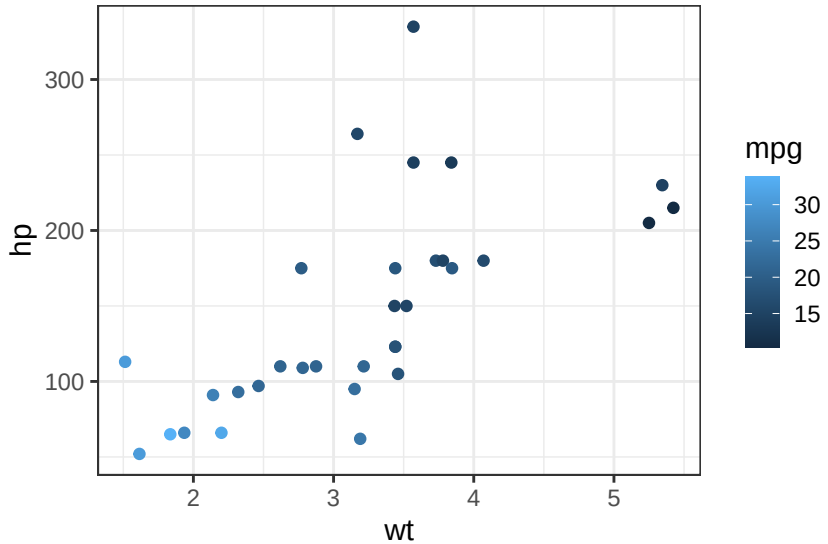
- Let's try to predict `mpg` with `hp` and `wt`

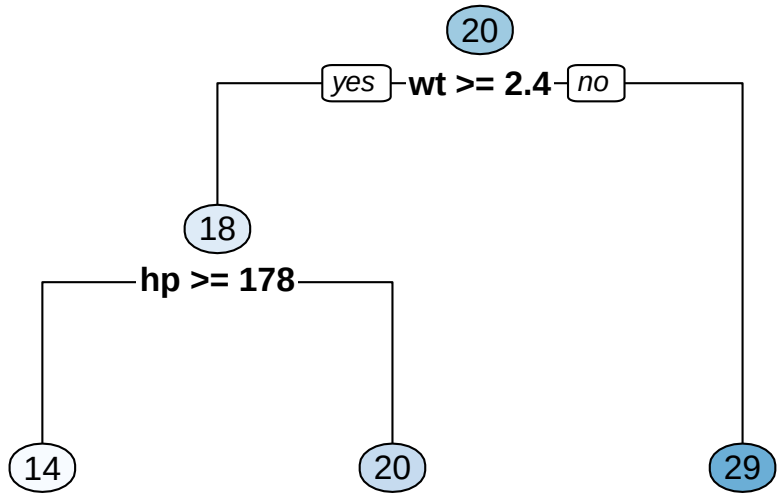


Extra Practice

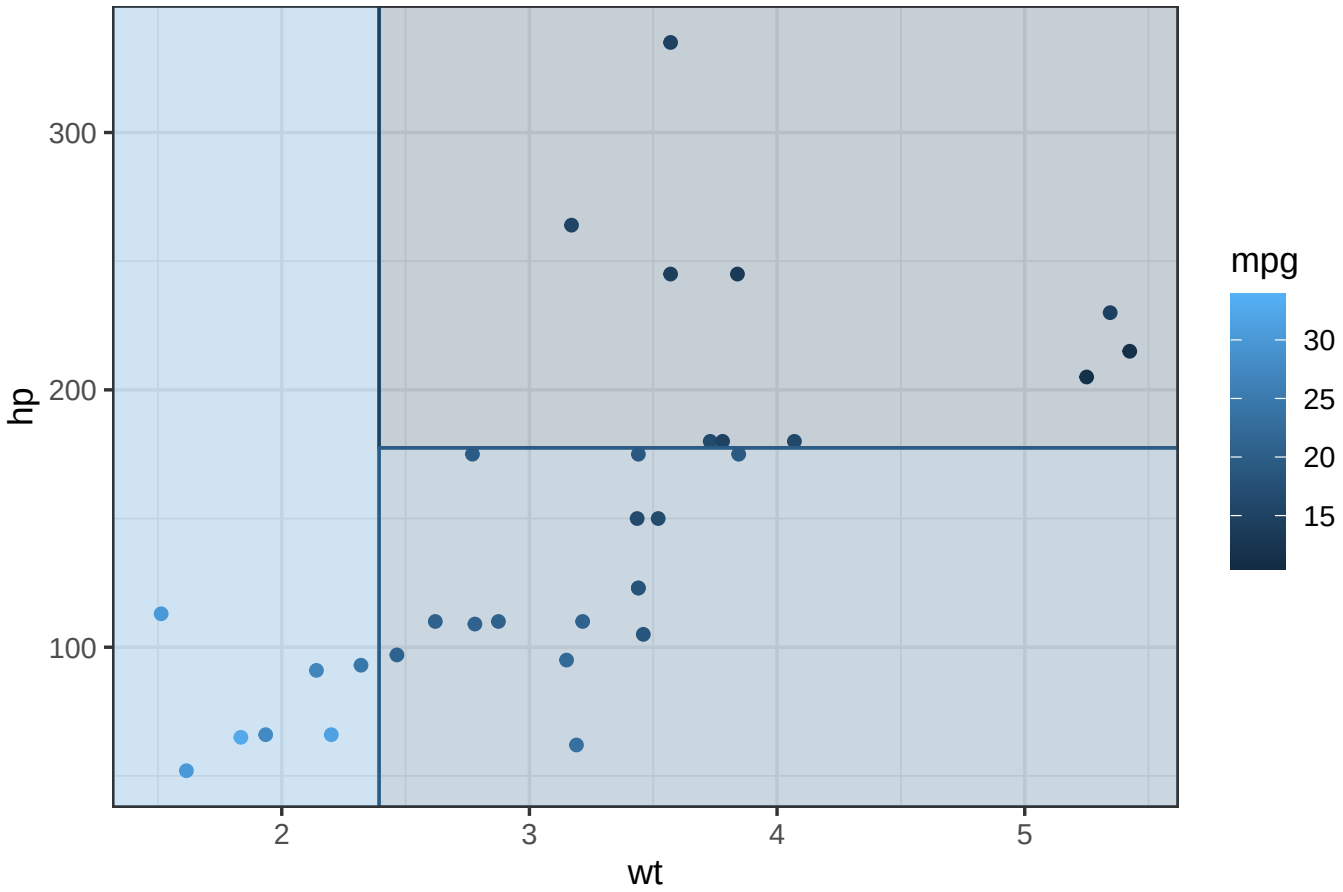
Task: With a partner:

- Divide the predictor space (on the left) into the groups that correspond to the following tree (on the right)
- Answer: If you were to build a multiple linear regression model that predicts `mpg` with `wt` and `hp`, what would you expect the *signs* of the slopes on `wt` and `hp` to be?





Results




```
##                Estimate Std. Error   t value     Pr(>|t|)
## (Intercept) 37.22727012 1.59878754 23.284689 2.565459e-20
## hp          -0.03177295 0.00902971 -3.518712 1.451229e-03
## wt          -3.87783074 0.63273349 -6.128695 1.119647e-06
```

Section 3

Recap

Recap

Today, we...

- Introduced Regression Trees
- Visualized, interpretted, and assessed trees

Next time, we will...

- Discuss tree "pruning" to avoid overfitting and improve accuracy